Compte rendu – TP1

systemd, réseau avancé et automatisation – projet « Hélios »
Administration d'un Linux Server – Bachelor Informatique 2e année (BTC2), IPSSI Lille

Étudiant	Bilal Chaybouti
Date	2 novembre 2026
Environnement	VirtualBox – Rocky Linux 9.8 Minimal `srv-core` (192.168.56.10) et `srv-edge` (192.168.56.11)
Résultat	Missions 1, 2, 3 et bonus réalisées – tous les critères de réussite validés

Sommaire et bilan

Critère de réussite (énoncé)	Preuve	Statut
helios-collect actif, enabled, redémarre seul après kill -9	§1.2, §1.3	✔
MemoryMax=100663296 via un drop-in	§1.3	✔
helios.target liste helios-collect	§1.4	✔
Bond active-backup, MII up, bascule ≤ 2 ICMP perdus	§2.1, §2.2	✔ **(2 perdus)**
Trames 802.1Q vlan 42 visibles dans tcpdump	§2.3	✔
Route depuis 10.42.42.10 via 10.42.42.11, sinon via la passerelle	§2.4	✔
Configuration réseau persistante après reboot	§2.5	✔
Script : codes retour 2 (verrou) et 3 (source absente)	§3.1	✔
Timer : prochaine exécution à 02:30	§3.2	✔
Bonus : path unit déclenchée par dépôt de fichier	§4	✔

Chiffres clés

Mesure	Avant	Après
Temps de boot total	1 min 43,820 s	**21,577 s** (≈ −80 %)
multi-user.target atteint	1 min 7,760 s	**5,760 s**
Score `systemd-analyze security`	5.3 MEDIUM	**0.6 SAFE**
Bascule du bond	180 ICMP envoyés, 178 reçus → 2 perdus	

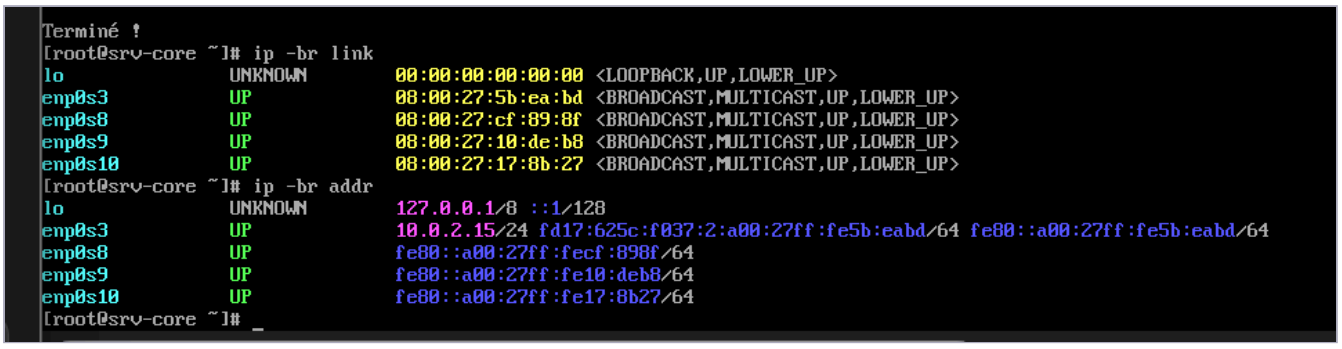
0. Mise en place de l'environnement

La VM `srv-core` a été créée sous VirtualBox (2 Go RAM, 2 vCPU, 20 Go, installation manuelle sans « unattended ») avec 4 cartes réseau :

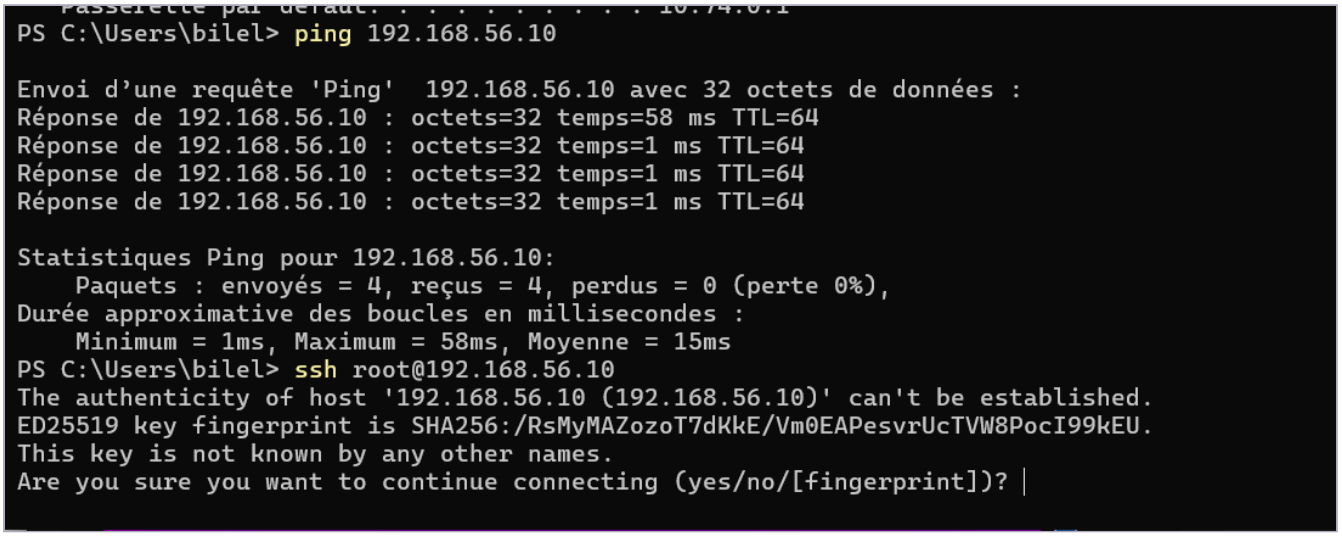
Carte VirtualBox	Interface Linux	Mode	Rôle
Adapter 1	enp0s3	NAT	Accès Internet (10.0.2.15)
Adapter 2	enp0s8	Réseau privé hôte (192.168.56.0/24)	Administration SSH depuis l'hôte
Adapter 3	enp0s9	Réseau interne `helios-lan`	Port 1 du bond
Adapter 4	enp0s10	Réseau interne `helios-lan`	Port 2 du bond

Le réseau host-only existant étant déjà utilisé par le lab Windows Server (192.168.10.0/24), un réseau dédié 192.168.56.1/24 a été créé, et `enp0s8` a reçu une IP fixe (pas de DHCP) :

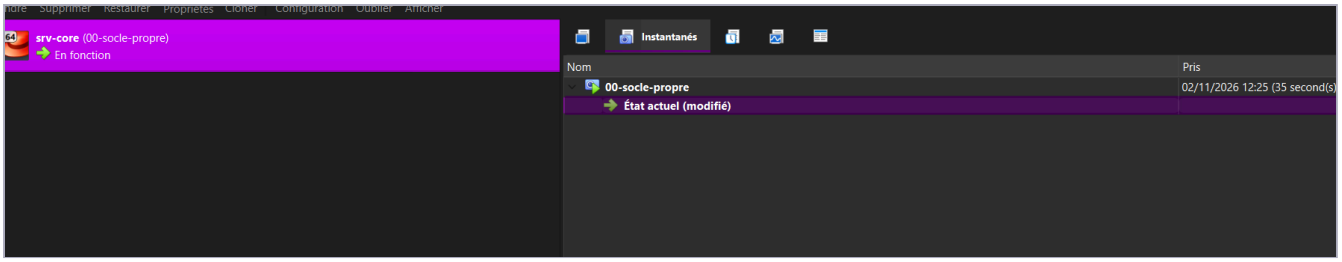
```
nmcli con mod enp0s8 ipv4.method manual ipv4.addresses 192.168.56.10/24 ipv4.never-default yes
connection.autoconnect yes
nmcli con up enp0s8
```



Rocky installé : les 4 interfaces enp0s3, enp0s8, enp0s9, enp0s10 sont présentes



Ping et connexion SSH depuis l'hôte Windows vers srv-core

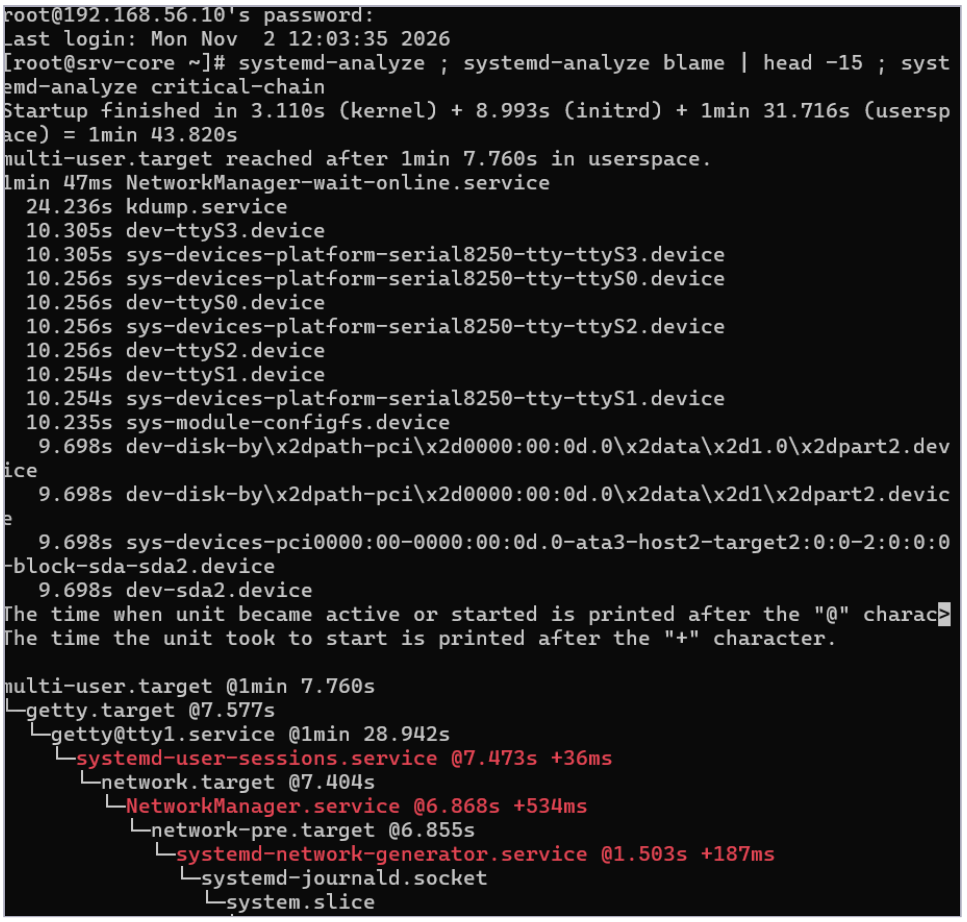


Instantané `00-socle-propre` pris avant le TP

Mission 1 – systemd

1.1 Analyse et optimisation du démarrage

```
systemd-analyze ; systemd-analyze blame | head -15 ; systemd-analyze critical-chain
```



Boot initial : 1 min 43,820 s

Analyse : deux services expliquent la lenteur :

- `NetworkManager-wait-online.service` (1 min 0,047 s) : il attend que toutes les interfaces soient configurées. enp0s9/enp0s10 n'ayant pas encore d'adresse, il attend jusqu'à son timeout.
- `kdump.service` (24,2 s) : préparation de la capture de crash du noyau, inutile sur une VM de TP.

```
systemd-analyze plot > /root/boot-avant.svg
systemctl disable --now firewalld          # temporaire, réactivé au TP3
systemctl mask NetworkManager-wait-online.service
systemctl disable --now kdump              # initiative personnelle justifiée (–24 s)
systemctl reboot
```

`mask` crée un lien vers `/dev/null` : le service ne peut plus être démarré, même comme dépendance. La désactivation de kdump ne figure pas dans l'énoncé : c'est une optimisation complémentaire, justifiée par la mesure.

```
[root@srv-core ~]# systemd-analyze plot > /root/boot-avant.svg
systemctl disable --now firewalld
systemctl mask NetworkManager-wait-online.service
systemctl disable --now kdump
systemctl reboot
Removed "/etc/systemd/system/multi-user.target.wants/firewalld.service".
Removed "/etc/systemd/system/dbus-org.fedoraproject.FirewallD1.service".
Created symlink /etc/systemd/system/NetworkManager-wait-online.service → /de
v/null.
Removed "/etc/systemd/system/multi-user.target.wants/kdump.service".
[root@srv-core ~]# Connection to 192.168.56.10 closed by remote host.
Connection to 192.168.56.10 closed.
PS C:\Users\bilel>
```

Désactivation de firewalld et kdump, masquage de wait-online

```
Connection to 192.168.56.10 closed.
PS C:\Users\bilel> ssh root@192.168.56.10
root@192.168.56.10's password:
Last login: Mon Nov  2 12:29:22 2026
[root@srv-core ~]# systemd-analyze ; systemd-analyze plot > /root/boot-apres
.svg
Startup finished in 3.860s (kernel) + 11.914s (initrd) + 5.802s (userspace)
= 21.577s
multi-user.target reached after 5.760s in userspace.
[root@srv-core ~]#
```

Après optimisation : 21,577 s (userspace 5,802 s)

Les graphiques `boot-avant.svg` et `boot-apres.svg` sont joints dans `captures/`.

1.2 Service de collecte helios-collect

Script en boucle écrivant toutes les *N* secondes une ligne `date,charge,mémoire disponible` dans `/var/lib/helios/telemetry.csv`, exécuté par un compte système dédié `helios` (sans shell). L'unit (`configs/helios-collect.service`) utilise :

- `Type=simple`, `Restart=on-failure`, `RestartSec=5s` : relance automatique en cas d'échec ;
- `After=/Wants=network-online.target` : ordre de démarrage ;
- un premier durcissement : `NoNewPrivileges`, `ProtectSystem=strict` + `ReadWritePaths`, `ProtectHome`, `PrivateTmp`…
- `CapabilityBoundingSet=` **volontairement vide** : le service ne dispose d'aucune capability Linux.

```
[Install]
WantedBy=multi-user.target
EOF
systemctl daemon-reload && systemctl enable --now helios-collect.service
systemctl status helios-collect.service --no-pager
Created symlink /etc/systemd/system/multi-user.target.wants/helios-collect.s
ervice → /etc/systemd/system/helios-collect.service.
● helios-collect.service - Collecteur de telemetrie Helios
     Loaded: loaded (/etc/systemd/system/helios-collect.service; enabled; pr
eset: disabled)
     Active: active (running) since Mon 2026-11-02 12:30:36 CET; 55ms ago
       Docs: https://wiki.helios.lan/collect
   Main PID: 945 ((llect.sh))
      Tasks: 1 (limit: 10637)
     Memory: 436.0K (peak: 712.0K)
        CPU: 17ms
     CGroup: /system.slice/helios-collect.service
             └─945 "(llect.sh)"

nov. 02 12:30:36 srv-core systemd[1]: Started Collecteur de telemetrie …ios.
Hint: Some lines were ellipsized, use -l to show in full.
[root@srv-core ~]#
```

helios-collect : active (running), enabled

```
nov. 02 12:30:36 srv-core systemd[1]: Started Collecteur de telemetrie …ios.
Hint: Some lines were ellipsized, use -l to show in full.
[root@srv-core ~]# sleep 20 ; cat /var/lib/helios/telemetry.csv
systemd-analyze security helios-collect.service | tail -3
2026-11-02T12:30:37+01:00,0.06,1459632
2026-11-02T12:30:52+01:00,0.05,1459436
2026-11-02T12:31:07+01:00,0.04,1459436
2026-11-02T12:31:22+01:00,0.03,1459436
✗ UMask=                                                  Files created
by service are world-readable by default                  0.1

→ Overall exposure level for helios-collect.service: 5.3 MEDIUM 😐
[root@srv-core ~]#
```

Données collectées toutes les 15 s – score de sécurité initial 5.3 MEDIUM

1.3 Redémarrage automatique, drop-in et durcissement

```
kill -9 "$(systemctl show -p MainPID --value helios-collect.service)"
sleep 8 ; systemctl status helios-collect.service --no-pager | head -5
```

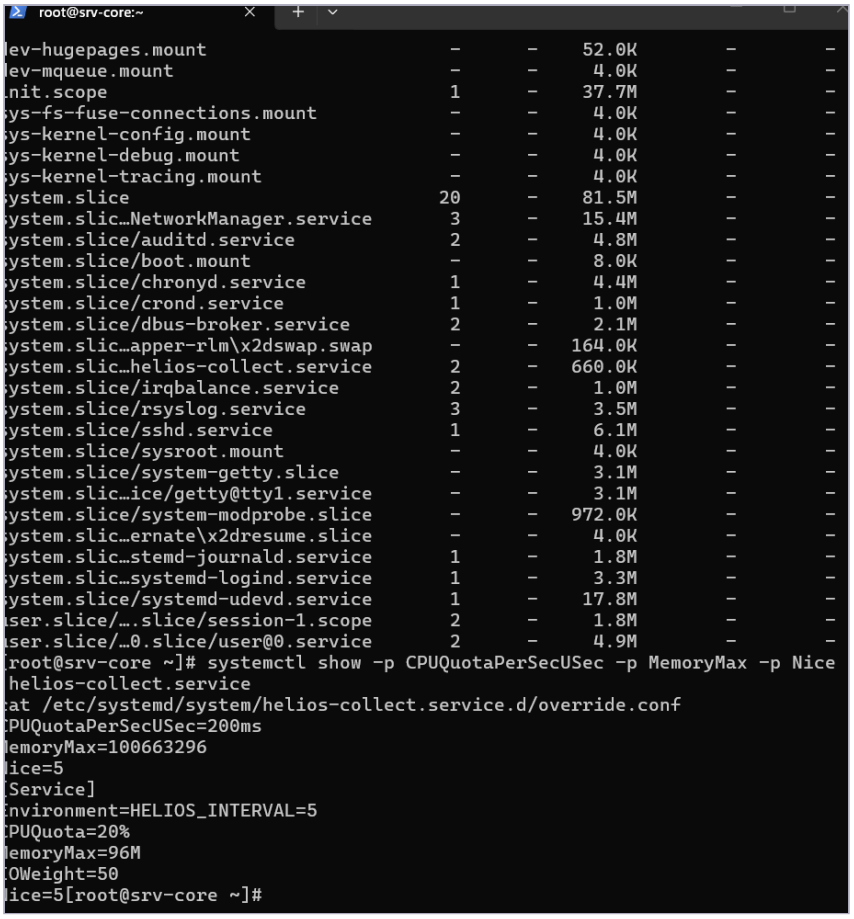
```
[root@srv-core ~]# kill -9 "$(systemctl show -p MainPID --value helios-colle
ct.service)"
sleep 8 ; systemctl status helios-collect.service --no-pager | head -5
● helios-collect.service - Collecteur de telemetrie Helios
     Loaded: loaded (/etc/systemd/system/helios-collect.service; enabled; pr
eset: disabled)
     Active: active (running) since Mon 2026-11-02 12:35:13 CET; 2s ago
       Docs: https://wiki.helios.lan/collect
   Main PID: 1045 (helios-collect.)
[root@srv-core ~]#
```

Après kill -9 : le service est relancé (nouveau PID 1045, « 2s ago »)

Un `SIGKILL` est considéré comme un échec ; `Restart=on-failure` relance donc le service après `RestartSec=5s`.

Drop-in (`systemctl edit`) : l'unit d'origine n'est jamais modifiée, la surcharge est placée dans `/etc/systemd/system/helios-collect.service.d/override.conf`.

```
[Service]
Environment=HELIOS_INTERVAL=5
CPUQuota=20%
MemoryMax=96M
IOWeight=50
Nice=5
```



CPUQuotaPerSecUSec=200ms, MemoryMax=100663296 (96 Mio), Nice=5

Durcissement (énoncé : « notez le score, puis durcissez »). Directives ajoutées au même drop-in, choisies car le collecteur ne fait que lire `/proc` et écrire dans `/var/lib/helios` :

Directives	Effet
PrivateDevices, PrivateNetwork, PrivateUsers	Aucun périphérique, aucun accès réseau, espace d'utilisateurs isolé
ProtectKernelModules/Logs, ProtectClock, ProtectHostname, ProtectProc=invisible	Interdit de charger des modules, lire les logs noyau, changer l'heure ou le nom d'hôte ; masque les autres processus
RestrictNamespaces, RestrictRealtime, RestrictAddressFamilies=AF_UNIX	Pas de nouveaux namespaces, pas d'ordonnancement temps réel, sockets locales uniquement

LockPersonality, MemoryDenyWriteExecute, SystemCallArchitectures=native, SystemCallFilter=@system-service	Limite les appels système à ceux d'un service classique, interdit la mémoire écrivable et exécutable
UMask=0027	Fichiers créés non lisibles par « les autres »

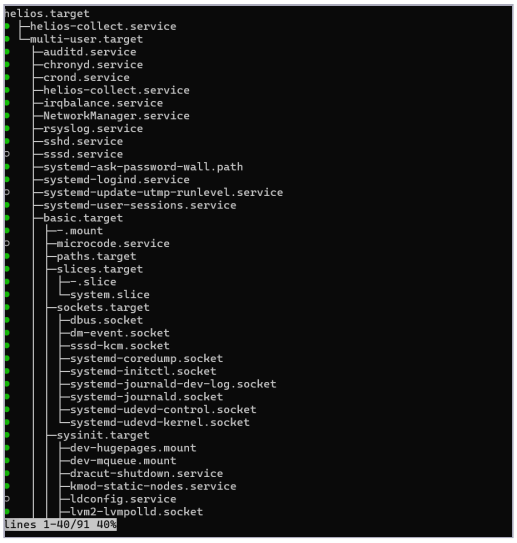


Service toujours fonctionnel (une ligne toutes les 5 s) – score passé à 0.6 SAFE

1.4 Cible métier et priorités de processus

`helios.target` regroupe les services de la pile Hélios afin de pouvoir les arrêter et démarrer ensemble (`systemctl stop/start helios.target`). `AllowIsolate=no` empêche de basculer tout le système sur cette cible.

```
systemctl add-wants helios.target helios-collect.service
systemctl enable helios.target ; systemctl list-dependencies helios.target
```



helios-collect.service apparaît sous helios.target

```
nice -n 19 sha256sum /dev/zero & NPID=$!
renice -n 5 -p "$NPID" ; ionice -c 2 -n 7 -p "$NPID" ; ionice -p "$NPID" ; chrt -p "$NPID"
```

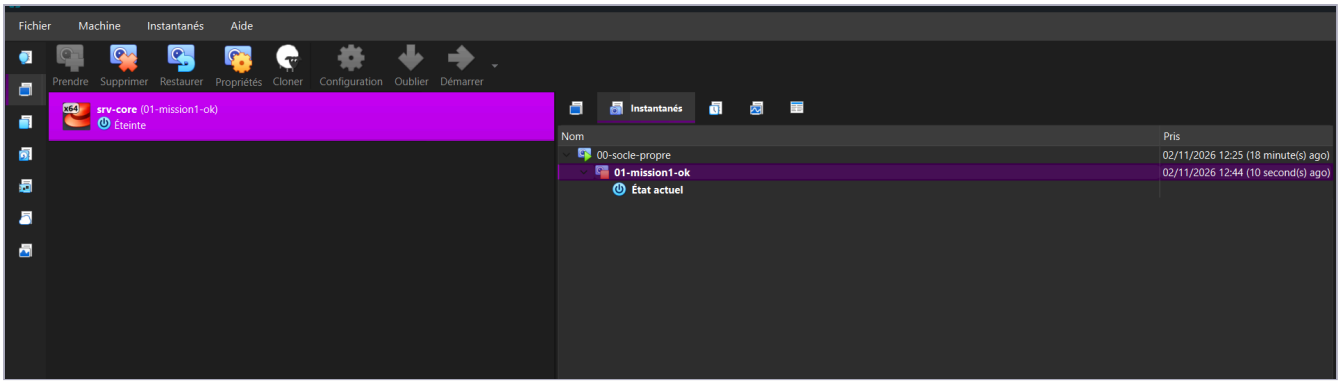
```
[root@srv-core ~]# nice -n 19 sha256sum /dev/zero & NPID=$!
ps -o pid,ni,pri,cls,rtprio,comm -p "$NPID"
renice -n 5 -p "$NPID" ; ionice -c 2 -n 7 -p "$NPID" ; ionice -p "$NPID" ; c
hrt -p "$NPID"
ps -o pid,ni,pri,cls,rtprio,comm -p "$NPID"
kill "$NPID"
[1] 1450
    PID  NI PRI CLS RTPRIO COMMAND
   1450  19   0  TS      - sha256sum
1450 (process ID) priorité précédente 19, nouvelle priorité 5
best-effort : priorité 7
stratégie d'ordonnancement actuelle pour le PID 1450 : SCHED_OTHER
priorité de planification actuelle pour le PID 1450 : 0
    PID  NI PRI CLS RTPRIO COMMAND
   1450   5  14  TS      - sha256sum
[root@srv-core ~]#
```

NI 19 → 5, ionice best-effort prio 7, politique SCHED_OTHER

Baisser la valeur nice (19 → 5) **augmente** la priorité CPU : seul root peut le faire. `ionice -c 2 -n 7` place le processus en classe d'E/S « best-effort » avec la priorité la plus basse. `chrt` confirme l'ordonnancement standard `SCHED_OTHER` (classe TS), non temps réel.

Mission 2 – Réseau avancé

`srv-edge` a été obtenue par clonage intégral de `srv-core` (nouvelles adresses MAC, adapter 4 désactivé), puis renommée (`hostnamectl set-hostname srv-edge`) avec l'IP d'administration 192.168.56.11. Instantané `01-mission1-ok` pris avant les manipulations réseau.



Instantanés de srv-core

> Préalable : l'installeur avait créé des profils automatiques sur enp0s9/enp0s10. Ils ont été supprimés (`nmcli con delete enp0s9 enp0s10`) sur les deux VM pour qu'ils ne reprennent pas les interfaces au redémarrage.

2.1 Agrégation active-backup

```
nmcli con add type bond con-name bond0 ifname bond0 bond.options "mode=active-
backup,miimon=100,primary=enp0s9,fail_over_mac=active" \
  ipv4.method manual ipv4.addresses 10.42.0.10/24 ipv4.never-default yes ipv6.method disabled
nmcli con add type ethernet con-name bond0-p1 ifname enp0s9 master bond0
nmcli con add type ethernet con-name bond0-p2 ifname enp0s10 master bond0
```

- `active-backup` : une seule carte transmet, l'autre est en secours (tolérance de panne, pas d'agrégation de débit) ;
- `miimon=100` : état du lien vérifié toutes les 100 ms ;
- `primary=enp0s9` : carte préférée, reprise automatique dès son retour ;
- `fail_over_mac=active` : le bond prend la MAC de la carte active, ce qui évite les problèmes d'apprentissage MAC dans VirtualBox.

```
[root@srv-core ~]# cat /proc/net/bonding/bond0 ; ip -d link show bond0
Ethernet Channel Bonding Driver: v5.14.0-687.61.1.el9_8.x86_64

Bonding Mode: fault-tolerance (active-backup) (fail_over_mac active)
Primary Slave: enp0s9 (primary_reselect always)
Currently Active Slave: enp0s9
MII Status: up
MII Polling Interval (ms): 100
Up Delay (ms): 0
Down Delay (ms): 0
Peer Notification Delay (ms): 0

Slave Interface: enp0s9
MII Status: up
Speed: 1000 Mbps
Duplex: full
Link Failure Count: 0
Permanent HW addr: 08:00:27:10:de:b8
Slave queue ID: 0

Slave Interface: enp0s10
MII Status: up
Speed: 1000 Mbps
Duplex: full
Link Failure Count: 0
Permanent HW addr: 08:00:27:17:8b:27
Slave queue ID: 0
6: bond0: <BROADCAST,MULTICAST,MASTER,UP,LOWER_UP> mtu 1500 qdisc noqueue st
ate UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:10:de:b8 brd ff:ff:ff:ff:ff:ff promiscuity 0 allmult
i 0 minmtu 68 maxmtu 65535
    bond mode active-backup active_slave enp0s9 miimon 100 updelay 0 downdel
ay 0 peer_notify_delay 0 use_carrier 1 arp_interval 0 arp_missed_max 2 arp_v
alidate none arp_all_targets any primary enp0s9 primary_reselect always fail
_over_mac active xmit_hash_policy layer2 resend_igmp 1 num_grat_arp 1 all_sl
aves_active 0 min_links 0 lp_interval 1 packets_per_slave 1 lacp_active on l
acp_rate slow ad_select stable tlb_dynamic_lb 1 addrgenmode none numtxqueues
 16 numrxqueues 16 gso_max_size 65536 gso_max_segs 65535 tso_max_size 65536
tso_max_segs 65535 gro_max_size 65536 gso_ipv4_max_size 65536 gro_ipv4_max_s
ize 65536
[root@srv-core ~]#
```

/proc/net/bonding/bond0 : active-backup, slave actif enp0s9, MII up partout

```
nmcli con mod enp0s8 ipv4.addresses 192.168.56.11/24
nmcli con up enp0s8
client_loop: send disconnect: Connection reset
PS C:\Users\bilel> ssh root@192.168.56.11
The authenticity of host '192.168.56.11 (192.168.56.11)' can't be establishe
d.
ED25519 key fingerprint is SHA256:/RsMyMAZozoT7dKkE/Vm0EAPesvrUcTVW8PocI99kE
U.
This host key is known by the following other names/addresses:
    C:\Users\bilel/.ssh/known_hosts:5: 192.168.56.10
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.11' (ED25519) to the list of known ho
sts.
root@192.168.56.11's password:
Permission denied, please try again.
root@192.168.56.11's password:
Last failed login: Mon Nov  2 12:52:34 CET 2026 from 192.168.56.1 on ssh:not
ty
There was 1 failed login attempt since the last successful login.
Last login: Mon Nov  2 12:51:27 2026 from 192.168.56.1
[root@srv-edge ~]# nmcli con delete enp0s9 enp0s10
Connexion « enp0s9 » (361d6bc9-82ae-30b3-ac52-228ef8a5a437) supprimée.
Connexion « enp0s10 » (4faccb26-0fe9-3413-b89e-37345d01f24a) supprimée.
[root@srv-edge ~]# nmcli con add type ethernet con-name metier ifname enp0s9
 ipv4.method manual ipv4.addresses 10.42.0.11/24 ipv4.never-default yes ipv6
.method disabled
nmcli con up metier
ping -c 3 10.42.0.10
Connexion « metier » (9eb55da5-c19d-4646-80a1-81262180a909) ajoutée avec suc
cès.
Connexion activée (chemin D-Bus actif : /org/freedesktop/NetworkManager/Acti
veConnection/10)
PING 10.42.0.10 (10.42.0.10) 56(84) octets de données.
64 octets de 10.42.0.10 : icmp_seq=1 ttl=64 temps=5.10 ms
64 octets de 10.42.0.10 : icmp_seq=2 ttl=64 temps=2.96 ms
64 octets de 10.42.0.10 : icmp_seq=3 ttl=64 temps=1.80 ms

--- statistiques ping 10.42.0.10 ---
3 paquets transmis, 3 reçus, 0% packet loss, time 2007ms
rtt min/avg/max/mdev = 1.801/3.287/5.100/1.366 ms
[root@srv-edge ~]#
```

srv-edge (10.42.0.11, profil « metier ») joint srv-core via le bond

2.2 Test de bascule

Ping continu de srv-core vers srv-edge, puis coupure du câble virtuel de l'adapter 3 (enp0s9) depuis l'hôte :

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" controlvm srv-core setlinkstate3 off
```

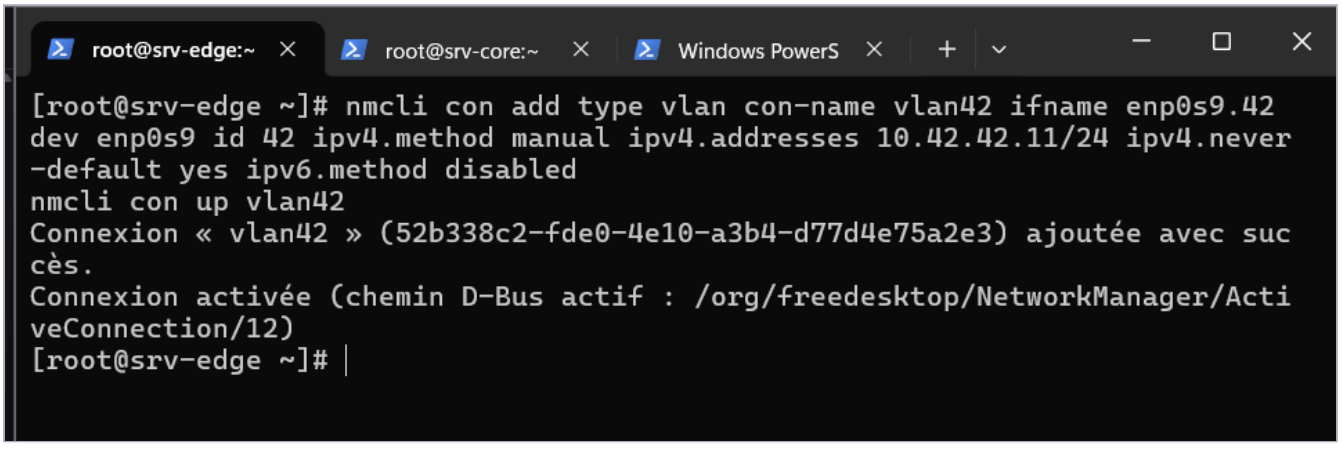


180 envoyés / 178 reçus (2 perdus) – enp0s10 devient actif – logs noyau de la bascule à 12:57:34

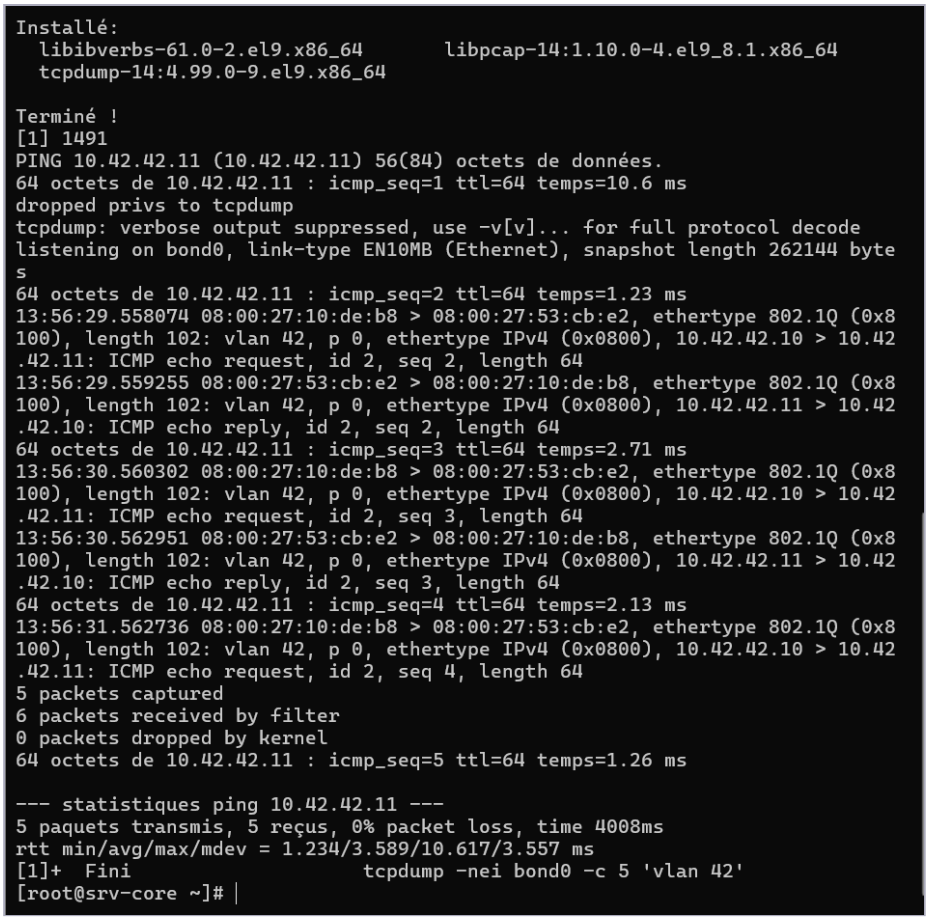
Le noyau détecte la perte de lien (« link status definitely down, disabling slave ») et bascule sur enp0s10 (« making interface the new active one »). **Seulement 2 requêtes ICMP perdues**, conforme au critère. Après `setlinkstate3 on`, enp0s9 redevient active grâce à `primary`.

2.3 VLAN 802.1Q « transport » (id 42)

```
# srv-core
nmcli con add type vlan con-name bond0.42 ifname bond0.42 dev bond0 id 42 ipv4.method manual
ipv4.addresses 10.42.42.10/24 ipv4.never-default yes ipv6.method disabled
# srv-edge
nmcli con add type vlan con-name vlan42 ifname enp0s9.42 dev enp0s9 id 42 ipv4.method manual
ipv4.addresses 10.42.42.11/24 ipv4.never-default yes ipv6.method disabled
```



Interface VLAN côté srv-edge

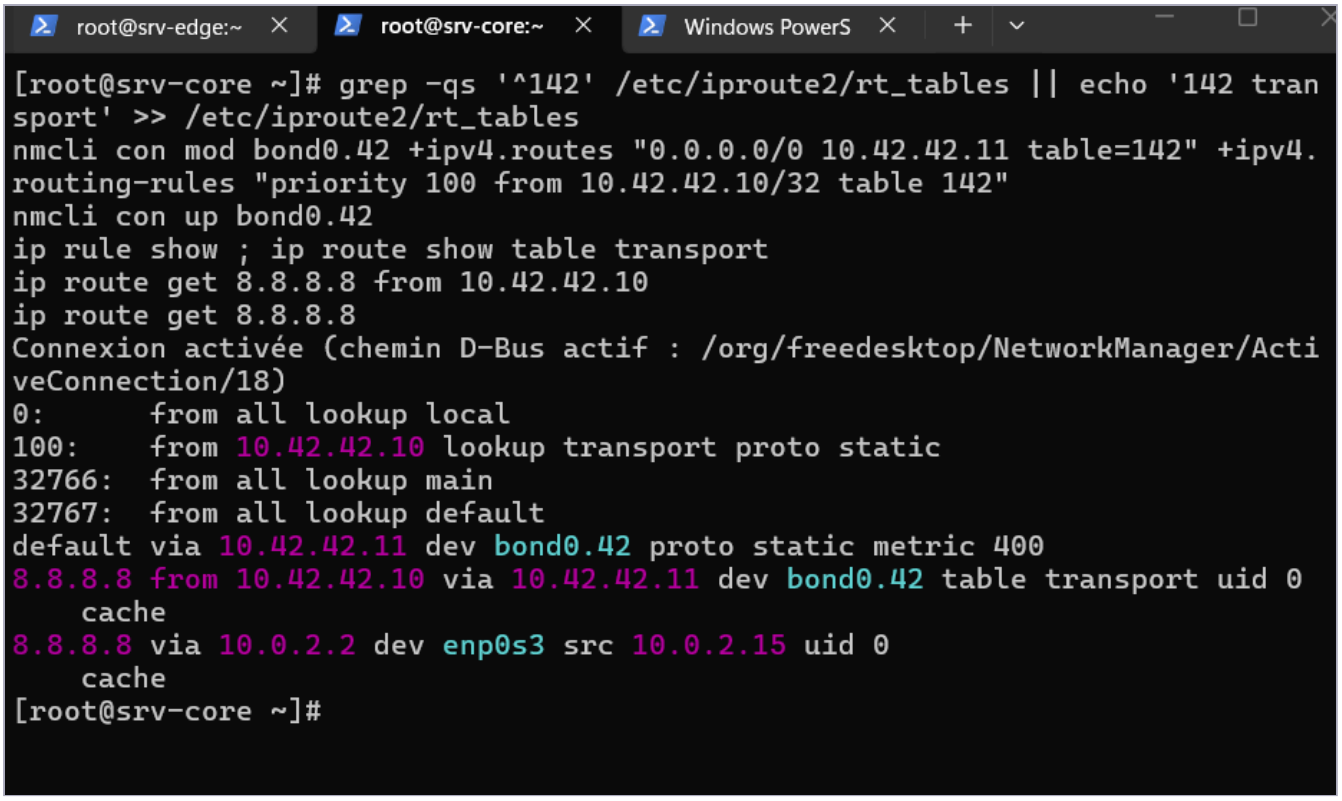


tcpdump : « ethertype 802.1Q (0x8100) … vlan 42 » sur chaque trame

Le VLAN est posé **au-dessus du bond** : il bénéficie donc aussi de la tolérance de panne. tcpdump montre l'en-tête 802.1Q (ethertype 0x8100, VID 42) dans les deux sens.

2.4 Routage par source (table « transport »)

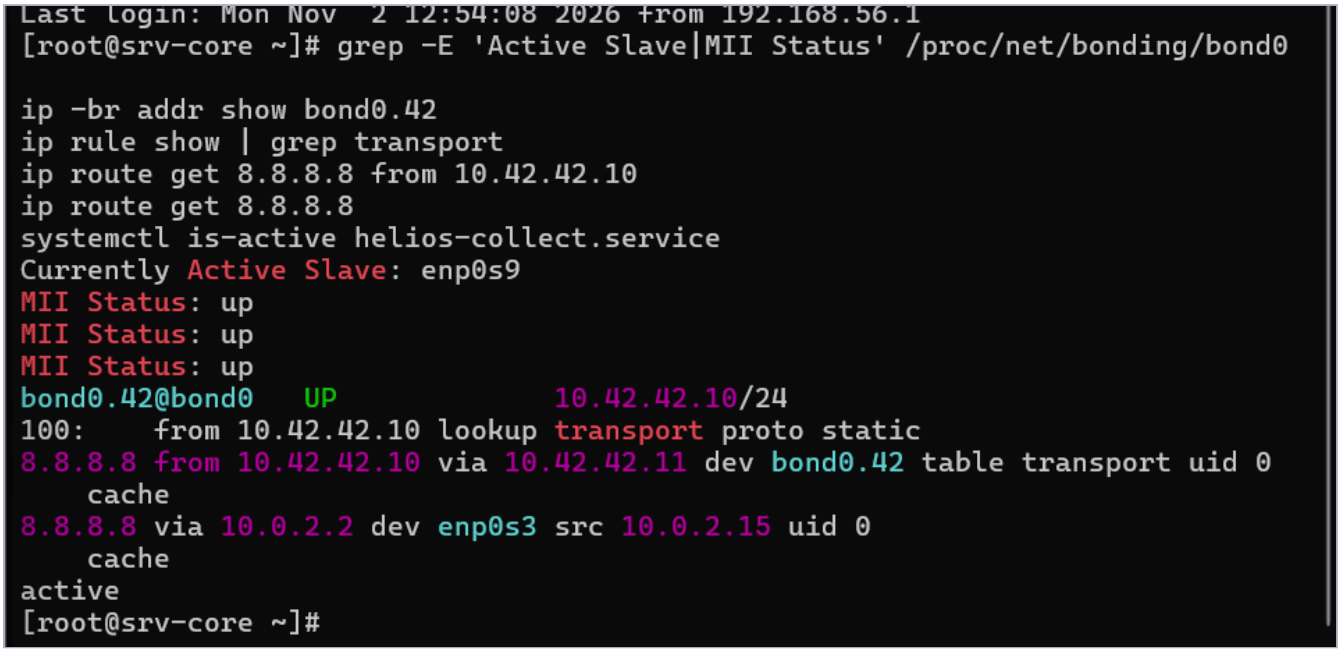
```
echo '142 transport' >> /etc/iproute2/rt_tables
nmcli con mod bond0.42 +ipv4.routes "0.0.0.0/0 10.42.42.11 table=142" \
  +ipv4.routing-rules "priority 100 from 10.42.42.10/32 table 142"
```



Règle 100 → table transport ; route selon l'adresse source

Le trafic émis **depuis** 10.42.42.10 consulte la table `transport` et sort via srv-edge (10.42.42.11) ; le reste suit la table `main` et sort par la passerelle NAT 10.0.2.2. Les routes et règles sont portées par le profil NetworkManager, donc persistantes.

2.5 Persistance après redémarrage



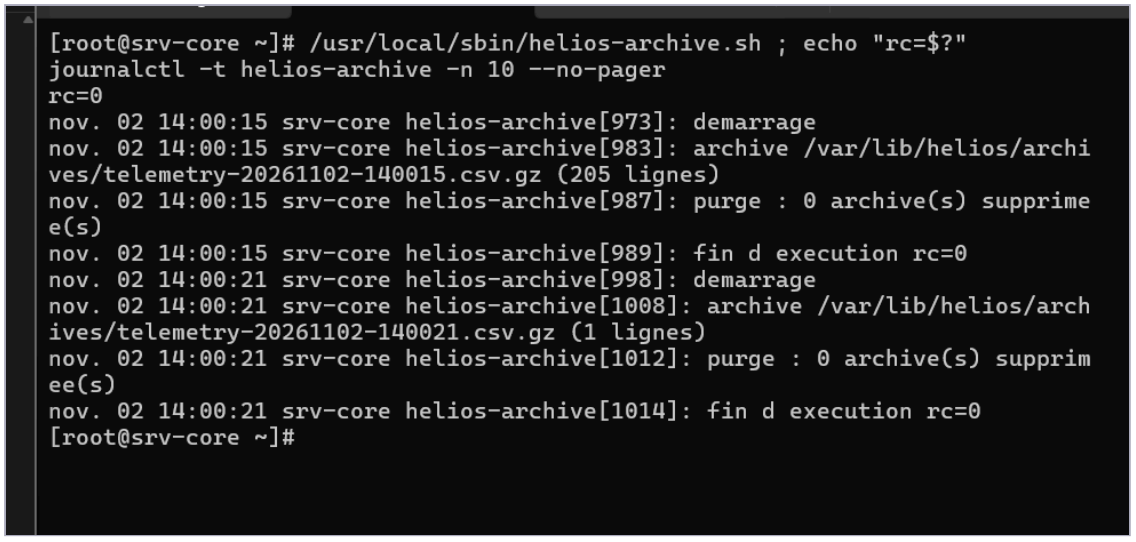
Après systemctl reboot : bond, VLAN, règle, routes et collecteur intacts

Mission 3 – Automatisation

3.1 Script helios-archive.sh

Le script (`configs/helios-archive.sh`) archive la télémétrie en `.csv.gz`, vide la source et purge les archives de plus de 14 jours. Points de robustesse :

- `set -euo pipefail` et `IFS` restreint : arrêt à la première erreur, variables non définies interdites ;
- `flock -n` sur `/run/lock/helios-archive.lock` : une seule instance à la fois (code 2 sinon) ;
- vérification des prérequis (`gzip`, source lisible) : code 3 ;
- `trap … EXIT` : nettoyage du répertoire temporaire et journalisation du code retour dans tous les cas ;
- journalisation via `logger -t helios-archive`, consultable avec `journalctl -t helios-archive`.



Code 0 : archive créée, purge, traces dans journald



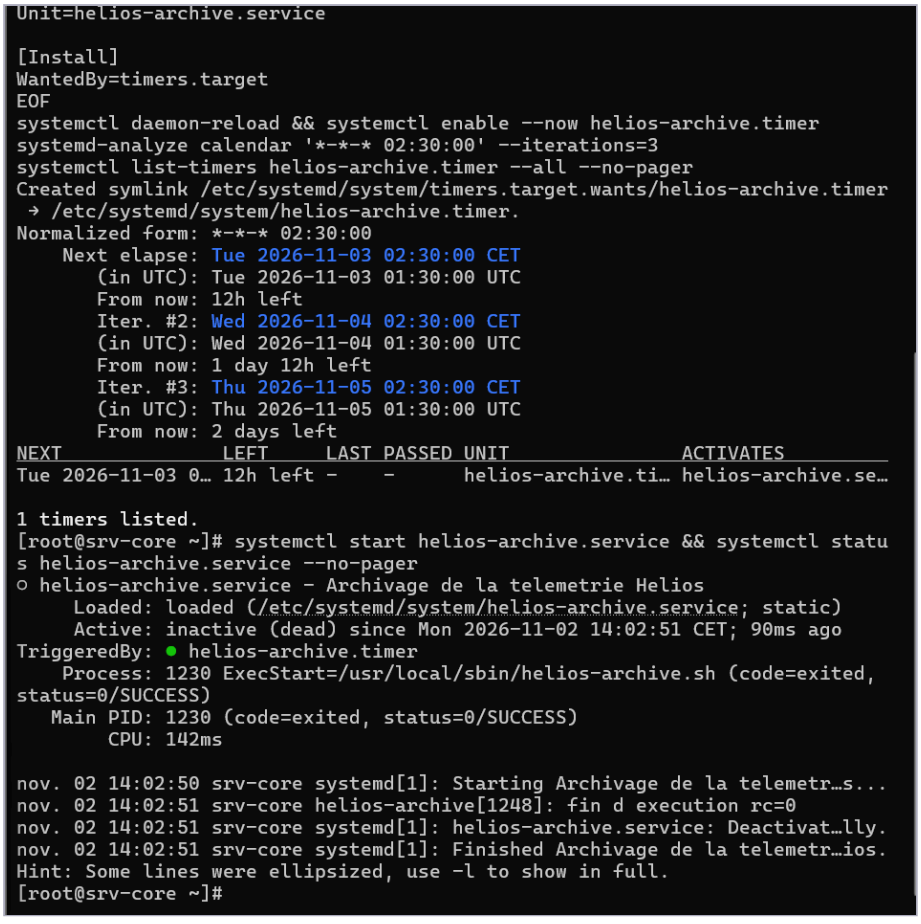
Code 2 : verrou déjà détenu par une autre instance

```
[root@srv-core ~]# mv /var/lib/helios/telemetry.csv /tmp/t.csv
/usr/local/sbin/helios-archive.sh ; echo "rc attendu 3 -> $?"
mv /tmp/t.csv /var/lib/helios/telemetry.csv
rc attendu 3 -> 3
[root@srv-core ~]#
```

Code 3 : fichier source absent

3.2 Timer systemd et comparaison avec cron

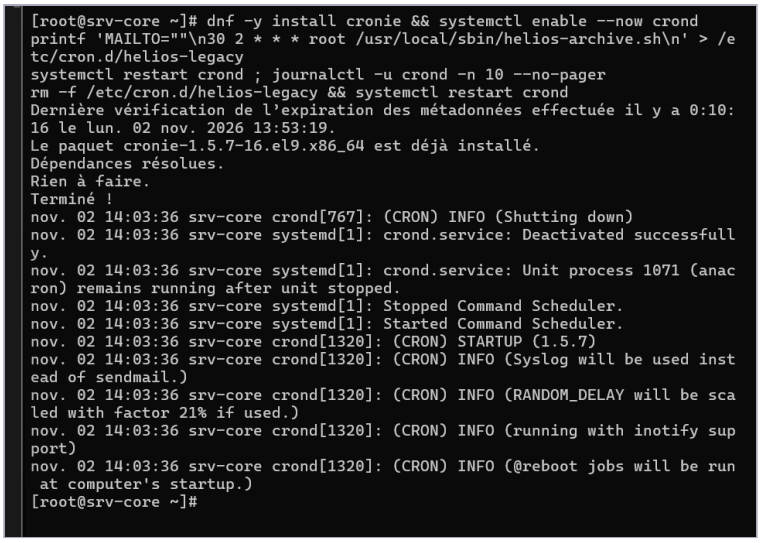
```
[Timer]
OnCalendar=*-*-* 02:30:00
AccuracySec=1min
RandomizedDelaySec=120
Persistent=yes
Unit=helios-archive.service
```



Prochaine exécution : mardi 3 nov. 02:30 CET – lancement manuel : status=0/SUCCESS

Le service est de type `oneshot` (exécution puis arrêt : « inactive (dead) » est normal), avec `Nice=10` et `IOSchedulingClass=idle` pour ne pas gêner la production.

```
printf 'MAILTO=""\n30 2 * * * root /usr/local/sbin/helios-archive.sh\n' > /etc/cron.d/helios-
legacy
systemctl restart crond ; journalctl -u crond -n 10 --no-pager
rm -f /etc/cron.d/helios-legacy && systemctl restart crond
```



Journal de crond : aucune information sur la tâche Hélios ni sur sa prochaine exécution

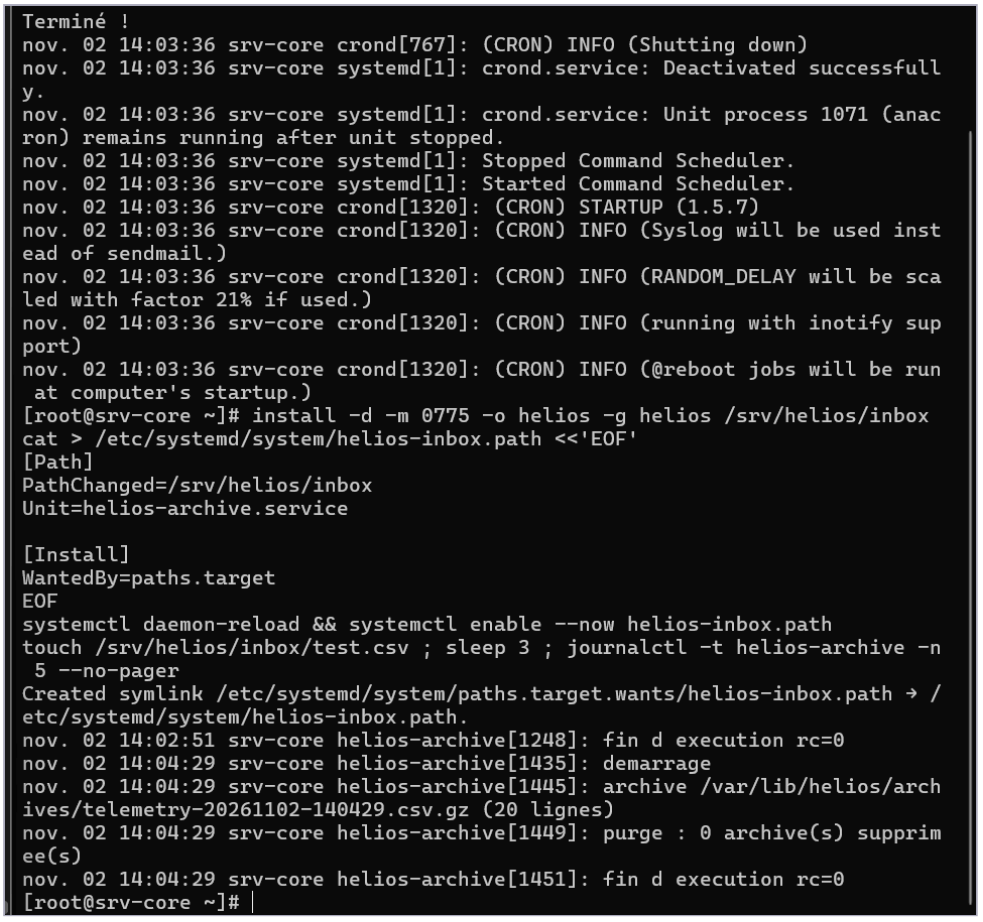
Critère	cron	Timer systemd
Rattrapage d'une exécution manquée (VM éteinte)	Non (sauf anacron, à la journée)	Oui : `Persistent=yes`
Journalisation	Syslog/mail, non lié à la tâche	journald, lié à l'unit (`systemctl status` , code retour)
Dépendances	Aucune	`After=` , `Requires=` , `Condition*=`
Limitation des ressources	Manuelle (nice dans la commande)	`Nice=` , `IOSchedulingClass=` , `CPUQuota=` , `MemoryMax=`
Étalement de la charge	Non (RANDOM_DELAY global)	`RandomizedDelaySec=` par timer
Prochaine exécution visible	Non	`systemctl list-timers`
Test manuel	Copier la commande	`systemctl start helios-archive.service`

Observation : dans le journal de crond, rien ne confirme que la tâche a été chargée ni quand elle s'exécutera, alors que `list-timers` donne directement l'échéance.

Bonus – Déclenchement par dépôt de fichier

```
[Path]
PathChanged=/srv/helios/inbox
Unit=helios-archive.service

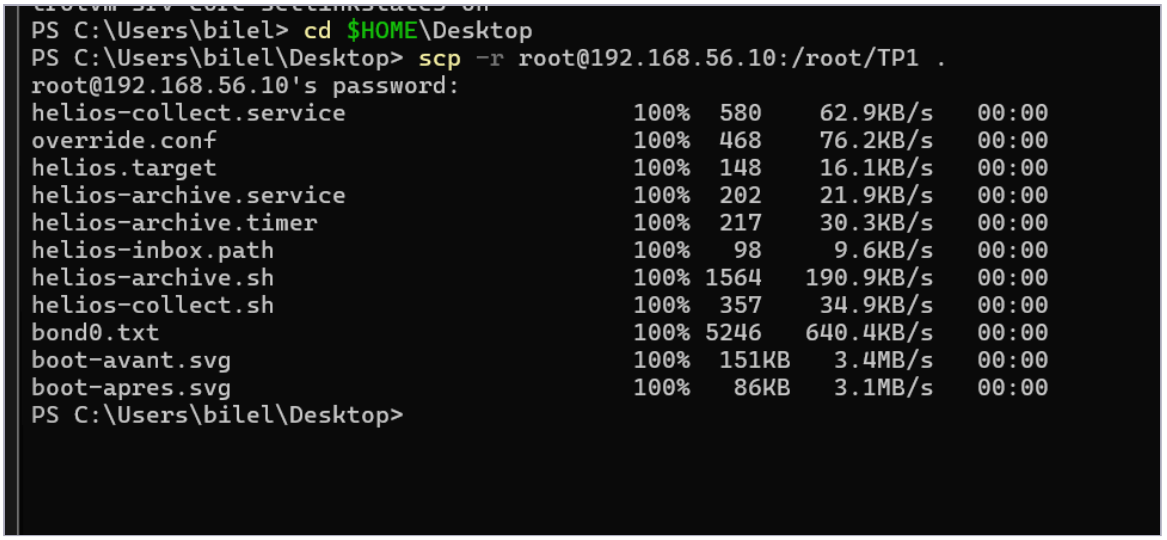
[Install]
WantedBy=paths.target
```



touch test.csv → archivage lancé automatiquement à 14:04:29 (rc=0)

Choix de `PathChanged` plutôt que `PathExistsGlob` : le script ne vide pas l'inbox. Avec `PathExistsGlob`, le fichier existerait toujours après l'exécution, et systemd relancerait le service en boucle jusqu'à atteindre la limite de démarrage (`unit-start-limit-hit`). `PathChanged` ne réagit qu'à une modification du répertoire.

Livrables et conclusion



Récupération des fichiers de configuration sur l'hôte (scp)

Dossier	Contenu
`configs/`	helios-collect.sh, helios-collect.service, override.conf, helios.target, helios-archive.sh, helios-archive.service, helios-archive.timer, helios-inbox.path, bond0.txt
`captures/`	boot-avant.svg, boot-apres.svg, captures d'écran
`RAPPORT.md`	Synthèse demandée par l'énoncé

Tous les objectifs sont atteints. Le TP montre l'intérêt de systemd au-delà du simple démarrage de services : mesure du boot, supervision avec relance automatique, limites de ressources par cgroups, isolation de sécurité (5.3 → 0.6), et planification plus traçable que cron. Côté réseau, la combinaison bond + VLAN + routage par source donne un lien tolérant aux pannes (2 paquets perdus à la bascule), segmenté et routé de façon sélective, entièrement persistant grâce à NetworkManager.

Difficultés rencontrées : lignes coupées dans le PDF de l'énoncé (corrigées : `--no-pager`, `-printf .`...), conflit d'adressage avec les réseaux host-only du lab Windows Server (résolu par un réseau dédié 192.168.56.0/24), et profils réseau créés automatiquement à l'installation sur enp0s9/enp0s10 (supprimés avant la création du bond).